

```
# =====
# 1. PARAMETERS & BALANCED INGREDIENT DATA
# =====
batch_size = 50.0

names = [
    "Maize", "Soya Hipro", "Full Fat Soya", "Maize Gluten", "Rice ODGS", "Meat & Bone",
    "Soya Oil", "Limestone", "DCP", "Salt (NaCl)", "L-Lysine", "DL-Methionine"
]

costs = np.array([20.0, 60.0, 52.0, 36.0, 60.0, 40.0, 60.0, 100.0, 3.0, 45.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0])

ME = np.array([3408, 3521, 2430, 2725, 3564, 2167, 2953, 8800, 0, 0, 0, 4000, 5000])
CP = np.array([8.18, 42.17, 51.78, 48.91, 62.77, 49.65, 57.39, 0, 0, 0, 0, 94.0, 58.0, 58.0, 58.0, 58.0, 58.0, 58.0, 58.0, 58.0])
Fat = np.array([4.18, 20.18, 1.73, 6.99, 6.88, 8.90, 99.13, 99.0, 0, 0, 0, 0, 0, 0])
Ca = np.array([0.02, 0.15, 0.31, 0.31, 0.03, 12.50, 4.90, 0, 38.0, 22.0, 0, 0, 0])
P = np.array([0.05, 0.175, 0.24, 0.24, 0.07, 3.60, 0.84, 0, 0, 18.0, 0, 0, 0])
Lys = np.array([0.218, 2.107, 2.748, 1.161, 0.847, 1.415, 1.083, 0, 0, 0, 0, 78.0, 0])
Met = np.array([0.158, 0.462, 0.606, 0.653, 1.359, 0.334, 0.592, 0, 0, 0, 0, 0, 99.0])
Na = np.array([0.02, 0.04, 0.02, 0.20, 0.01, 0.55, 0.45, 0, 0, 0, 39.3, 0, 0])
Cl = np.array([0.04, 0.03, 0.05, 0.45, 0.05, 0.69, 0.51, 0, 0, 0, 60.7, 0, 0])

nutrients = [CP, ME, Fat, Ca, P, Lys, Met, Na, Cl]
nutrient_names = ['Protein (%)', 'ME (kcal/kg)', 'Fat (%)', 'Calcium (%)', 'Phosphorus (%)']

L_bounds = [
    np.array([23.0, 21.0, 20.0]), np.array([2950, 3000, 3050]), np.array([3.5, 3.5, 3.5]),
    np.array([0.9, 0.9, 0.92]), np.array([0.4, 0.45, 0.36]), np.array([1.2, 1.11, 1.11]),
    np.array([0.5, 0.6, 0.5]), np.array([0.2, 0.17, 0.17]), np.array([0.19, 0.17, 0.17])
]

T_bounds = [
    np.array([24.0, 22.0, 21.0]), np.array([3000, 3050, 3150]), np.array([4.0, 4.0, 4.0]),
    np.array([7.0, 1.0, 1.0]), np.array([0.45, 0.46, 0.4]), np.array([1.3, 1.12, 1.12]),
    np.array([0.6, 0.62, 0.59]), np.array([0.22, 0.22, 0.22]), np.array([0.24, 0.24, 0.24])
]

steam_cost_kg = 40.0 / 1000
water_cost_kg = 2.9 / 1000
electricity_cost_kg = 138.0 / 1000
labor_cost_kg = 63.0 / 1000
packaging_cost_kg = 189.5 / 1000

pdi_coef = [137.683, -0.298, 0.482, -0.016, -1.752]

# =====
# 2. ERGONOMICS (DYNAMIC LI)
# =====
worker_data = [
    {"id": "PACKING 1", "H": 25, "V": 100, "D": 0, "A": 0, "F": 4, "C": 0.95},
    {"id": "PACKING 2", "H": 25, "V": 110, "D": 0, "A": 90, "F": 4, "C": 0.95},
    {"id": "LOADING 3", "H": 0, "V": 170, "D": 0, "A": 0, "F": 4, "C": 0.95},
    {"id": "LOADING 4", "H": 0, "V": 170, "D": 100, "A": 0, "F": 4, "C": 0.95},
    {"id": "LIFTING 5", "H": 20, "V": 150, "D": 150, "A": 45, "F": 4, "C": 0.95},
    {"id": "CUTTING 6", "H": 18, "V": 80, "D": 0, "A": 0, "F": 4, "C": 0.95}
]

def calculate_rwl(H, V, D, A, F, C):
    HM = max(0.5, min(1.0, 25 / H if 0 < H <= 63 else 0.5))
```

```

VM = max(0.5, min(1.0, 1 - (0.003 * abs(V - 75))))
DM = max(0.5, min(1.0, 0.82 + (4.5 / D) if D > 0 else 1.0))
AM = max(0.5, min(1.0, 1 - (0.0032 * A)))
FM = 0.84 if F == 4 else 1.0
return 23.0 * HM * VM * DM * AM * FM * C

worker_rwls = [calculate_rwl(w['H'], w['V'], w['D'], w['A'], w['F'], w['C']) for w in worker_data]

# =====
# 3. DECOUPLED STAGE-BY-STAGE OPTIMIZATION
# =====
max_p = [0.65, 0.35, 0.30, 0.15, 0.15, 0.08, 0.10, 0.05, 0.03, 0.03, 0.01, 0.015, 0.02]

bnds_single = []
for mp in max_p: bnds_single.append((0.001, batch_size * mp))
bnds_single.extend([(75, 85), (2, 4), (500, 700), (6, 12), (25, 50)])

x_opt = np.zeros((3, 13))
T_opt, M_opt, PS_opt, R_opt, W_opt, a_PDI = np.zeros(3), np.zeros(3), np.zeros(3), np.zeros(3), np.zeros(3), np.zeros(3)
total_cost_all = 0
success_flags = []

L_PDI, T_PDI_arr = np.array([85.0, 87.0, 86.0]), np.array([90.0, 92.0, 91.0])

for s in range(3):
    def objective_single(vars):
        x = np.array(vars[0:13])
        T, M, PS, R, W = vars[13], vars[14], vars[15], vars[16], vars[17]

        # Priority 1: Nutrition
        nutr_penalty = 0
        nutr_sats = []
        for i in range(len(nutrients)):
            val = (x @ nutrients[i]) / batch_size
            L, U = L_bounds[i][s], T_bounds[i][s]

            if val < L: nutr_penalty += (L - val)**2 * 100000
            if val > U: nutr_penalty += (val - U)**2 * 100000

            if U > L:
                sat = 1.0 - abs(val - ((L+U)/2)) / ((U-L)/2)
                nutr_sats.append(np.clip(sat, 0, 1))
            else:
                nutr_sats.append(1.0 if abs(val - L) < 1e-4 else 0.0)

        lam1 = np.mean(nutr_sats)

        # Priority 2: PDI
        pdi_val = pdi_coef[0] + pdi_coef[1]*T + pdi_coef[2]*M + pdi_coef[3]*PS + pdi_coef[4]*R + pdi_coef[5]*W
        lam2 = np.clip((pdi_val - L_PDI[s]) / (T_PDI_arr[s] - L_PDI[s]), 0, 1)

        # Priority 3: Cost (UPDATED to Rs 42-48/kg bounds -> 2100 to 2400 per 50kg)
        c_raw = np.sum(x * costs)
        c_proc = steam_cost_kg*T*batch_size + water_cost_kg*M*batch_size + electric_cost_kg*PS*batch_size
        c_lab = labor_cost_kg * batch_size * (10 / max(1, R))
        c_pack = packaging_cost_kg * W
        c_tot = c_raw + c_proc + c_lab + c_pack
        lam3 = np.clip((2400.0 - c_tot) / (2400.0 - 2100.0), 0, 1)

        # Priority 4: Ergonomics
        avg_li = np.mean([W / rwl for rwl in worker_rwls])
        lam4 = np.clip((1.8 - avg_li) / (1.8 - 0.8), 0, 1)

        return -(10000*lam1 + 1000*lam2 + 100*lam3 + 10*lam4) + nutr_penalty

```

```

cons_single = [{'type': 'eq', 'fun': lambda v: np.sum(v[0:13]) - batch_size}]

base_guess = [30.0, 5.0, 5.0, 2.0, 2.0, 2.0, 2.0, 0.5, 0.5, 0.5, 0.1, 0.2, 0.2]
guess = [g * (batch_size / sum(base_guess)) for g in base_guess] + [80.0, 3.0,

res = minimize(objective_single, guess, method='SLSQP', bounds=bnds_single, con

success_flags.append(res.success)
x_opt[s] = res.x[0:13]
T_opt[s], M_opt[s], PS_opt[s], R_opt[s], W_opt[s] = res.x[13:18]
a_PDI[s] = pdi_coef[0] + pdi_coef[1]*T_opt[s] + pdi_coef[2]*M_opt[s] + pdi_coef

c_raw = np.sum(x_opt[s] * costs)
c_proc = steam_cost_kg*T_opt[s]*batch_size + water_cost_kg*M_opt[s]*batch_size
c_lab = labor_cost_kg * batch_size * (10 / max(1, R_opt[s]))
c_pack = packaging_cost_kg * W_opt[s]
total_cost_all += (c_raw + c_proc + c_lab + c_pack)

stages = ['Pre-Starter (0-12d)', 'Starter (12-23d)', 'Finisher (23-45d)']

# =====
# 4. CLEAN MANAGEMENT REPORT OUTPUT
# =====
print("\n" + "="*80)
print(" LFGP BROILER FEED OPTIMIZATION: DECOUPLED STAGE ANALYSIS REPORT")
print("="*80)
print(f" Optimization Status : {'ALL SUCCESS' if all(success_flags) else 'CHECK LIM
print(f" Total System Cost   : Rs. {total_cost_all:,.2f} | Avg: Rs. {total_cost_a

for s in range(3):
    print("\n" + "-"*80)
    print(f" STAGE: {stages[s].upper()}")
    print("-"*80)

    print(" [PRIORITY 1: NUTRITIONAL ADEQUACY]")
    print(f" {'Nutrient':<15} | {'Actual':<8} | {'Target Range':<15} | {'Status'}")
    print(" " + "-"*65)
    for i in range(len(nutrients)):
        val = (x_opt[s] @ nutrients[i]) / batch_size
        L, U = L_bounds[i][s], T_bounds[i][s]

        if val < L - 0.01: status = "X BELOW TARGET"
        elif val > U + 0.01: status = "X ABOVE TARGET"
        else: status = "✓ WITHIN RANGE"
        print(f" {nutrient_names[i]:<15} | {val:>6.3f} | {L:>5.3f} - {U:<5.3f}

    print("\n [PRIORITY 2: PELLET QUALITY (PDI)]")
    pdi_status = "✓ TARGET MET" if a_PDI[s] >= T_PDI_arr[s] - 0.1 else "X BELOW TA
    print(f" Achieved PDI   : {a_PDI[s]:.2f}% (Target: {T_PDI_arr[s]:.1f}%) -> {pdi

    print("\n [PRIORITY 3: COST & PROCESS SETTINGS]")
    stage_c = np.sum(x_opt[s]*costs) + (steam_cost_kg*T_opt[s]*batch_size + water_c
    print(f" Stage Cost      : Rs. {stage_c:.2f} (Rs. {stage_c/batch_size:.2f} / kg)
    print(f" Machine Setting: Temp: {T_opt[s]:.1f}°C | Moist: {M_opt[s]:.1f}% | Siz

    print("\n" + "="*80)
    print(" [PRIORITY 4: ERGONOMIC SAFETY ANALYSIS (NIOSH LI)]")
    print("="*80)
    print(f" Optimized Bag Weight (Ws): {np.mean(W_opt):.1f} kg")
    print(f" {'Worker Task':<15} | {'RWL (kg)':<10} | {'Avg LI':<8} | {'Risk Level'}")
    print(" " + "-"*65)
    for i, w in enumerate(worker_data):
        avg_li = np.mean([W_opt[s] / worker_rwls[i] for s in range(3)])

```

```

risk = "● LOW" if avg_li <= 0.8 else "● MODERATE" if avg_li <= 1.0 else "● HIGH"
print(f" {w['id']:<15} | {worker_rwls[i]:>6.1f} kg | {avg_li:>6.2f} | {risk}")

# --- NEW: Ergonomic Deep Dive Appendix ---
print("\n" + "="*80)
print(" [APPENDIX: ERGONOMIC MATHEMATICAL DEEP DIVE (WORKER 1 & WORKER 4)]")
print("="*80)
opt_w = np.mean(W_opt)
rwl_w1 = worker_rwls[0]
rwl_w4 = worker_rwls[3]
print(f" Why did the optimizer reject the baseline 50.0 kg bag and choose {opt_w:.1f} kg?")
print("\n SCENARIO A: Factory Baseline (50.0 kg)")
print(f" • Worker 1 (Packing) LI : 50.0 / {rwl_w1:.1f} = {50.0/rwl_w1:.2f}")
print(f" • Worker 4 (Loading) LI : 50.0 / {rwl_w4:.1f} = {50.0/rwl_w4:.2f} (Catastrophic)")
avg_li_50 = np.mean([50.0 / rwl for rwl in worker_rwls])
print(f" • Global Average LI : {avg_li_50:.2f}")
lam4_50 = (1.8 - avg_li_50) / (1.8 - 0.8)
print(f" • Fuzzy Satisfaction λ4 : (1.8 - {avg_li_50:.2f}) / (1.0) = {lam4_50:.2f}")

print(f"\n SCENARIO B: Optimizer Compromise ({opt_w:.1f} kg)")
print(f" • Worker 1 (Packing) LI : {opt_w:.1f} / {rwl_w1:.1f} = {opt_w/rwl_w1:.2f}")
print(f" • Worker 4 (Loading) LI : {opt_w:.1f} / {rwl_w4:.1f} = {opt_w/rwl_w4:.2f}")
avg_li_opt = np.mean([opt_w / rwl for rwl in worker_rwls])
print(f" • Global Average LI : {avg_li_opt:.2f}")
lam4_opt = (1.8 - avg_li_opt) / (1.8 - 0.8)
print(f" • Fuzzy Satisfaction λ4 : (1.8 - {avg_li_opt:.2f}) / (1.0) = {lam4_opt:.2f}")
print("\n CONCLUSION:")
print(" Even at the optimized compromise of 40 kg, the physical hazard still violates")
print(" the maximum risk tolerance (LI = 1.8). The model proves that weight reduction")
print(" cannot solve the problem; physical workstation geometry must be redesigned.")
print("="*80 + "\n")

# =====
# 5. DASHBOARD PLOTTING
# =====
fig, axs = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Broiler Feed Formulation Dashboard (LFGP Final Architecture)', fontsize=14)
colors = plt.cm.tab20(np.linspace(0, 1, 13))

for s in range(3):
    labels = [names[i] for i in range(13) if x_opt[s, i] > 0.1]
    sizes = [x_opt[s, i] for i in range(13) if x_opt[s, i] > 0.1]
    cols = [colors[i] for i in range(13) if x_opt[s, i] > 0.1]
    axs[0, s].pie(sizes, labels=labels, colors=cols, autopct='%1.1f%%', startangle=90)
    axs[0, s].set_title(f'{stages[s]}\nRecipe Formulation', fontweight='bold')

c_raw_all = np.sum(x_opt @ costs)
c_proc_all = np.sum(steam_cost_kg*T_opt*batch_size + water_cost_kg*M_opt*batch_size)
c_lab_all = np.sum(labor_cost_kg * batch_size * (10 / np.maximum(1, R_opt)))
c_pack_all = np.sum(packaging_cost_kg * W_opt)
cost_vals = [c_raw_all, c_proc_all, c_lab_all, c_pack_all]
axs[1, 0].pie(cost_vals, labels=['Raw Material', 'Energy', 'Labor', 'Packaging'], autopct='%1.1f%%')
axs[1, 0].set_title('Global Cost Breakdown', fontweight='bold')

axs[1, 1].bar(['Temp (°C)', 'Moist (%)', 'Size (μm/10)', 'Rate (t/h)'], [np.mean(T_opt), np.mean(M_opt), np.mean(W_opt), np.mean(R_opt)])
axs[1, 1].set_title('Average Machine Settings', fontweight='bold')

worker_ids = [w['id'] for w in worker_data]
li_scores = [np.mean([W_opt[s] / worker_rwls[i] for s in range(3)]) for i in range(4)]
bar_colors = ['#2ecc71' if li<=0.8 else '#f1c40f' if li<=1.0 else '#e67e22' if li<=1.2 else '#9b59b6']
bars_li = axs[1, 2].bar(worker_ids, li_scores, color=bar_colors, edgecolor='black')

axs[1, 2].axhline(0.8, color='green', linestyle='--', label='Ideal (0.8)')
axs[1, 2].axhline(1.2, color='red', linestyle='--', label='Max (1.2)')

```

```
axs[1, 2].set_ylabel('Lifting Index (LI)')
axs[1, 2].set_title(f'Worker Lifting Risk (Avg Bag = {np.mean(W_opt):.1f}kg)', font
axs[1, 2].legend()

for bar in bars_li:
    yval = bar.get_height()
    axs[1, 2].text(bar.get_x() + bar.get_width()/2, yval + 0.05, f'{yval:.2f}', ha=

plt.tight_layout()
plt.show()
```

=====

LFGP BROILER FEED OPTIMIZATION: DECOUPLED STAGE ANALYSIS REPORT

=====

Optimization Status : CHECK LIMITS (See Maize Constraint)
Total System Cost : Rs. 5,639.81 | Avg: Rs. 37.60 / kg

STAGE: PRE-STARTER (0-12D)

[PRIORITY 1: NUTRITIONAL ADEQUACY]

Nutrient	Actual	Target Range	Status
Protein (%)	24.576	23.000 - 24.000	X ABOVE TARGET
ME (kcal/kg)	3054.727	2950.000 - 3000.000	X ABOVE TARGET
Fat (%)	10.732	3.500 - 4.000	X ABOVE TARGET
Calcium (%)	1.579	0.900 - 7.000	✓ WITHIN RANGE
Phosphorus (%)	0.501	0.400 - 0.450	X ABOVE TARGET
Lysine (%)	1.120	1.200 - 1.300	X BELOW TARGET
Methionine (%)	0.679	0.500 - 0.600	X ABOVE TARGET
Sodium (%)	0.183	0.200 - 0.220	X BELOW TARGET
Chloride (%)	0.280	0.190 - 0.240	X ABOVE TARGET

[PRIORITY 2: PELLET QUALITY (PDI)]

Achieved PDI : 91.67% (Target: 90.0%) -> ✓ TARGET MET

[PRIORITY 3: COST & PROCESS SETTINGS]

Stage Cost : Rs. 1872.90 (Rs. 37.46 / kg)
Machine Setting: Temp: 80.0°C | Moist: 3.0% | Size: 600µm | Rate: 8.0 t/h

STAGE: STARTER (12-23D)

[PRIORITY 1: NUTRITIONAL ADEQUACY]

Nutrient	Actual	Target Range	Status
Protein (%)	24.625	21.000 - 22.000	X ABOVE TARGET
ME (kcal/kg)	3059.749	3000.000 - 3050.000	X ABOVE TARGET
Fat (%)	10.798	3.500 - 4.000	X ABOVE TARGET
Calcium (%)	1.583	0.900 - 1.000	X ABOVE TARGET
Phosphorus (%)	0.502	0.450 - 0.460	X ABOVE TARGET
Lysine (%)	1.141	1.110 - 1.120	X ABOVE TARGET
Methionine (%)	0.705	0.600 - 0.620	X ABOVE TARGET
Sodium (%)	0.177	0.170 - 0.220	✓ WITHIN RANGE
Chloride (%)	0.271	0.170 - 0.240	X ABOVE TARGET

[PRIORITY 2: PELLET QUALITY (PDI)]

Achieved PDI : 92.24% (Target: 92.0%) -> ✓ TARGET MET

[PRIORITY 3: COST & PROCESS SETTINGS]

Stage Cost : Rs. 1880.51 (Rs. 37.61 / kg)
Machine Setting: Temp: 79.5°C | Moist: 3.1% | Size: 600µm | Rate: 7.8 t/h

STAGE: FINISHER (23-45D)

[PRIORITY 1: NUTRITIONAL ADEQUACY]

Nutrient	Actual	Target Range	Status
Protein (%)	24.041	20.000 - 21.000	X ABOVE TARGET
ME (kcal/kg)	3155.894	3050.000 - 3150.000	X ABOVE TARGET
Fat (%)	10.631	4.000 - 4.500	X ABOVE TARGET
Calcium (%)	1.454	0.920 - 1.000	X ABOVE TARGET
Phosphorus (%)	0.466	0.360 - 0.400	X ABOVE TARGET
Lysine (%)	1.132	1.000 - 1.100	X ABOVE TARGET

Methionine (%)		0.724		0.500 - 0.590		X ABOVE TARGET
Sodium (%)		0.155		0.170 - 0.220		X BELOW TARGET
Chloride (%)		0.236		0.170 - 0.240		✓ WITHIN RANGE

[PRIORITY 2: PELLET QUALITY (PDI)]
Achieved PDI : 91.67% (Target: 91.0%) -> ✓ TARGET MET

[PRIORITY 3: COST & PROCESS SETTINGS]
Stage Cost : Rs. 1886.40 (Rs. 37.73 / kg)
Machine Setting: Temp: 80.0°C | Moist: 3.0% | Size: 600µm | Rate: 8.0 t/h

[PRIORITY 4: ERGONOMIC SAFETY ANALYSIS (NIOSH LI)]

Optimized Bag Weight (Ws): 39.9 kg

Worker Task		RWL (kg)		Avg LI		Risk Level
PACKING 1		17.0 kg		2.35		● VERY HIGH
PACKING 2		11.7 kg		3.41		● VERY HIGH
LOADING 3		6.6 kg		6.08		● VERY HIGH
LOADING 4		5.7 kg		7.03		● VERY HIGH
LIFTING 5		10.3 kg		3.86		● VERY HIGH
CUTTING 6		18.1 kg		2.21		● VERY HIGH

[APPENDIX: ERGONOMIC MATHEMATICAL DEEP DIVE (WORKER 1 & WORKER 4)]

Why did the optimizer reject the baseline 50.0 kg bag and choose 39.9 kg instead?

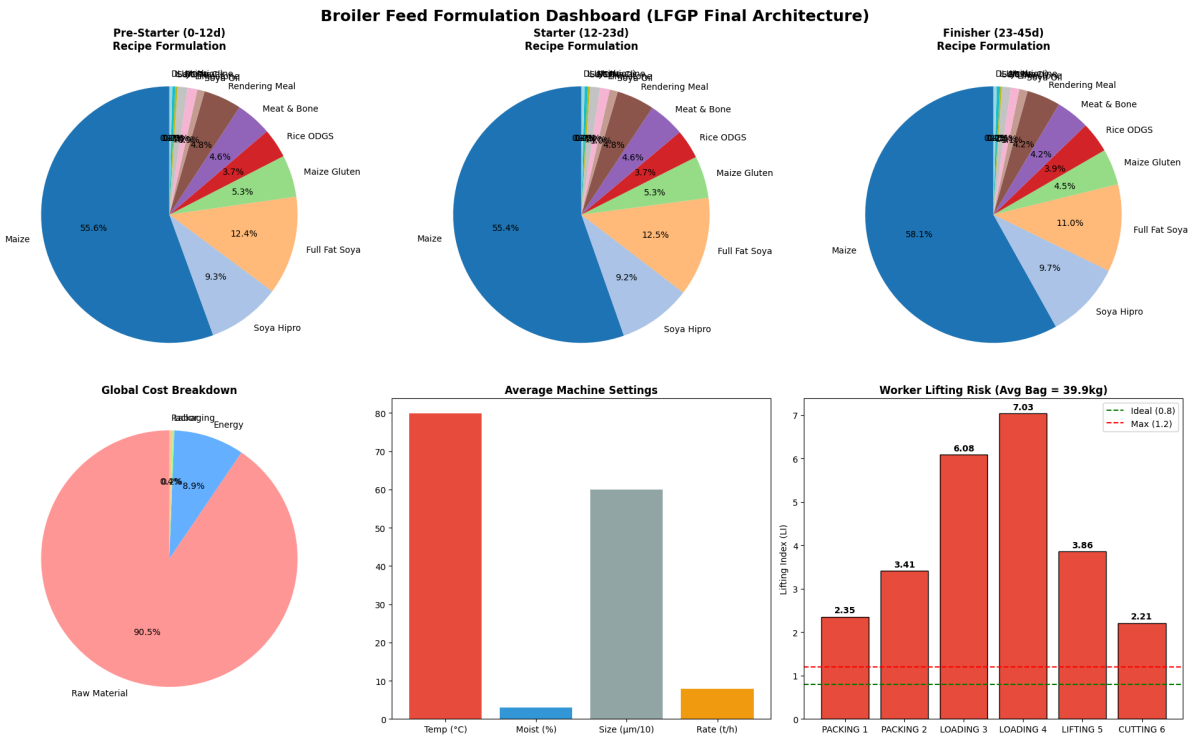
SCENARIO A: Factory Baseline (50.0 kg)

- Worker 1 (Packing) LI : $50.0 / 17.0 = 2.95$
- Worker 4 (Loading) LI : $50.0 / 5.7 = 8.81$ (Catastrophic Risk)
- Global Average LI : 5.21
- Fuzzy Satisfaction λ_4 : $(1.8 - 5.21) / (1.0) = -3.41$ -> mathematically clipped to 0.0

SCENARIO B: Optimizer Compromise (39.9 kg)

- Worker 1 (Packing) LI : $39.9 / 17.0 = 2.35$
- Worker 4 (Loading) LI : $39.9 / 5.7 = 7.03$
- Global Average LI : 4.16
- Fuzzy Satisfaction λ_4 : $(1.8 - 4.16) / (1.0) = -2.36$ -> clipped to 0.0

CONCLUSION:
Even at the optimized compromise of 40 kg, the physical hazard still violently exceeds the maximum risk tolerance (LI = 1.8). The model proves that weight reduction alone cannot solve the problem; physical workstation geometry must be redesigned.



In []: